

Day 21.1 – Jenkins CI/CD for GenAI: Docker, ECR, Helm and EKS

1. What we are building

For your capstone, Jenkins automates this flow:

```
Developer pushes code
↓
Jenkins Pipeline starts
↓
Backend lint + unit tests
Frontend lint + unit tests
↓
RAG golden-dataset evaluation
↓
Build backend and frontend Docker images
↓
Tag images with commit/build number
↓
Push images to Amazon ECR
↓
Deploy to development EKS using Helm
↓
Run health checks and RAG smoke tests
↓
Manual production approval
↓
Deploy the same images to production
↓
Production smoke tests
```

The important principle is:

Build once, test once, and promote the same immutable container images through environments.

Do not rebuild the application for production after testing a different development image.

2. Jenkins fundamentals

Jenkins controller and agents

Modern Jenkins terminology uses **controller** instead of master.

The controller:

- Stores Jenkins configuration.
- Schedules pipeline jobs.
- Manages credentials and plugins.
- Tracks build history.
- Coordinates agents.

Agents:

- Execute pipeline steps.
- Run tests.
- Build Docker images.
- Execute AWS CLI, Helm and `kubectl`.
- Can be static virtual machines or dynamically created Kubernetes pods.

Production builds should normally run on agents rather than on the controller's built-in node, primarily for security, scalability and resource isolation. ([Jenkins](#))

For this pipeline, an agent needs:

```
Git
Python
Node.js and npm
Docker
AWS CLI
kubectl
Helm
```

A better production pattern is to create a versioned Jenkins agent image containing these tools instead of manually installing them on every machine.

Declarative versus Scripted Pipeline

Declarative Pipeline

Declarative Pipeline has a predefined, opinionated structure:

```
pipeline {
  agent any

  stages {
    stage('Test') {
      steps {
        sh 'pytest'
      }
    }
  }
}
```

Advantages:

- Easier to read and review.
- Built-in support for stages, when, environment, options and post.
- Good for most application CI/CD pipelines.
- Easier for a larger team to maintain.

Scripted Pipeline

Scripted Pipeline is a more flexible Groovy-based pipeline:

```
node {
    stage('Test') {
        sh 'pytest'
    }
}
```

It provides more programming flexibility, but complex Scripted Pipelines can become difficult to understand. Declarative syntax is usually the preferred starting point, while `script {}` blocks or Shared Libraries can handle exceptional complexity. Jenkins supports both styles on the same underlying Pipeline system. ([Jenkins](#))

For your capstone, use **Declarative Pipeline**.

3. Pipeline anatomy

A basic Declarative Pipeline contains:

```
pipeline {
    agent any

    environment {
        APP_NAME = 'genai-app'
    }

    stages {
        stage('Test') {
            steps {
                sh 'run-tests.sh'
            }
        }

        stage('Build') {
            steps {
                sh 'docker build ...'
            }
        }

        stage('Deploy') {
            steps {
                sh 'helm upgrade --install ...'
            }
        }
    }

    post {
        always {
            echo 'Pipeline completed'
        }
    }
}
```

The main concepts are:

Element	Purpose
pipeline	Defines the complete pipeline
agent	Selects where the pipeline runs
environment	Defines environment variables
stages	Groups major pipeline phases
stage	Represents one logical phase
steps	Commands executed inside a stage
when	Controls whether a stage runs
post	Actions after success or failure
parameters	Runtime inputs such as production promotion

A Jenkinsfile should be stored in source control alongside the application. This gives the pipeline version history, code review and an audit trail. ([Jenkins](#))

4. Recommended repository structure

```

genai-platform/
├── Jenkinsfile
├── backend/
│   ├── app/
│   ├── tests/
│   ├── scripts/
│   │   ├── run_golden_eval.py
│   │   └── smoke_rag.py
│   ├── requirements.txt
│   ├── requirements-dev.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   ├── package.json
│   └── Dockerfile
├── evaluation/
│   └── golden-dataset.jsonl
├── deploy/
│   └── helm/
│       └── genai-app/
│           ├── Chart.yaml
│           ├── values.yaml
│           ├── values-dev.yaml
│           ├── values-prod.yaml
│           └── templates/
│               ├── backend-deployment.yaml
│               ├── backend-service.yaml
│               ├── frontend-deployment.yaml
│               ├── frontend-service.yaml
│               └── ingress.yaml

```

5. Pipeline stage design

Stage 1: Checkout

Jenkins retrieves the commit that triggered the pipeline.

```
checkout scm
```

In a Multibranch Pipeline, Jenkins can automatically discover branches and pull requests containing a Jenkinsfile. ([Jenkins](#))

Stage 2: Initialize image tag

Use an immutable image tag:

```
<jenkins-build-number>--<git-commit-sha>
```

Example:

```
184-a91f72c315e4
```

Backend image:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/rag-backend:184-a91f72c315e4
```

Frontend image:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/chat-frontend:184-a91f72c315e4
```

Avoid deploying only a mutable tag such as:

```
latest
```

A commit-based tag lets you answer:

- Which code is running?
- Which pipeline created the image?
- Which image should be rolled back?
- Are development and production using the same artifact?

For stronger traceability, record the ECR image digest as well.

Stage 3: Backend lint and tests

Typical backend checks:

```
ruff check .  
pytest  
mypy app
```

For the FastAPI RAG backend, unit tests should cover:

- API request validation.

- Authentication and authorization.
- Chunking logic.
- Metadata filtering.
- Retriever behavior.
- Prompt construction.
- LLM-provider adapters.
- Tool-calling validation.
- Error and retry handling.

External LLM and vector-database calls should normally be mocked in fast unit tests.

Stage 4: Frontend lint and tests

Typical frontend checks:

```
npm ci
npm run lint
npm run test:ci
npm run build
```

Tests can cover:

- Chat-message rendering.
- Streaming-token handling.
- Citation rendering.
- API errors.
- Authentication flow.
- Conversation-history state.
- Empty and loading states.

`npm ci` is generally preferable in CI because it installs dependencies from the lock file without modifying it.

Stage 5: RAG golden tests

Traditional unit tests cannot fully determine whether a RAG answer is useful.

A golden dataset might look like:

```
{
  "question": "How long are customer audit logs retained?",
  "expected_sources": ["security-policy.pdf"],
  "required_facts": ["seven years"]
}
```

The CI evaluation can measure:

- Whether the expected document was retrieved.
- Retrieval recall or hit rate.
- Whether required facts appear in the answer.
- Citation correctness.

- Groundedness.
- Refusal when supporting context is unavailable.
- Latency and token usage.

A simplified promotion gate could be:

Retrieval hit rate	$\geq 90\%$
Answer correctness	$\geq 85\%$
Citation validity	$\geq 95\%$
Unsupported-answer rate	$\leq 2\%$
P95 latency	≤ 5 seconds

Use deterministic retrieval settings and low-temperature generation where possible. LLM-based metrics should usually be combined with rule-based checks because model judging can vary.

Stage 6: Build Docker images

Build separate images for the backend and frontend:

```
docker build -t "${BACKEND_IMAGE}" backend/
docker build -t "${FRONTEND_IMAGE}" frontend/
```

Recommended practices:

- Use multi-stage Dockerfiles.
- Pin important dependency versions.
- Run containers as non-root users.
- Include OCI labels such as commit SHA.
- Keep model files out of the image unless intentionally bundled.
- Use `.dockerignore`.
- Scan images before deployment.
- Do not put API keys into Docker build arguments or image layers.

Jenkins can also run stages inside containerized execution environments, which helps standardize tools across agents. ([Jenkins](#))

Stage 7: Authenticate and push to ECR

ECR authentication:

```
aws ecr get-login-password --region "$AWS_REGION" |
docker login \
  --username AWS \
  --password-stdin "$ECR_REGISTRY"
```

Push both images:

```
docker push "$BACKEND_IMAGE"
docker push "$FRONTEND_IMAGE"
```

Amazon ECR uses `aws ecr get-login-password` to obtain a registry authentication token, after which Docker can push or pull images according to the IAM principal's permissions. AWS recommends least-privilege repository permissions rather than broad ECR access. ([AWS Documentation](#))

Stage 8: Connect Jenkins to EKS

The agent creates a temporary kubeconfig:

```
aws eks update-kubeconfig \
  --region "$AWS_REGION" \
  --name "$EKS_CLUSTER" \
  --kubeconfig "$WORKSPACE/.kubeconfig"
```

`kubectl` and Helm then use that kubeconfig.

EKS authentication is based on the AWS identity currently available to the AWS CLI. The identity needs AWS permission to describe the cluster and Kubernetes authorization to perform the required deployment operations. ([AWS Documentation](#))

For a private EKS API endpoint, the Jenkins agent must also have network connectivity to the cluster, usually by running inside the relevant VPC.

Stage 9: Helm deployment

Development:

```
helm upgrade --install genai-dev deploy/helm/genai-app \
  --namespace genai-dev \
  --create-namespace \
  -f deploy/helm/genai-app/values-dev.yaml \
  --set backend.image.repository="$BACKEND_REPOSITORY" \
  --set-string backend.image.tag="$IMAGE_TAG" \
  --set frontend.image.repository="$FRONTEND_REPOSITORY" \
  --set-string frontend.image.tag="$IMAGE_TAG" \
  --wait \
  --timeout 10m \
  --rollback-on-failure
```

Production:

```
helm upgrade --install genai-prod deploy/helm/genai-app \
  --namespace genai-prod \
  --create-namespace \
  -f deploy/helm/genai-app/values-prod.yaml \
  --set backend.image.repository="$BACKEND_REPOSITORY" \
  --set-string backend.image.tag="$IMAGE_TAG" \
  --set frontend.image.repository="$FRONTEND_REPOSITORY" \
  --set-string frontend.image.tag="$IMAGE_TAG" \
  --wait \
  --timeout 10m \
  --rollback-on-failure
```


`helm upgrade --install` creates the release when it does not exist and upgrades it when it already exists. Current Helm documentation provides `--rollback-on-failure`; Helm 3 pipelines also commonly use `--atomic` for wait-and-rollback behavior. Pin the Helm version in your agent image and use the corresponding supported flag. (helm.sh)

6. Development versus production

There are two common designs.

Option A: Same cluster, separate namespaces

```
EKS cluster: genai-eks
├─ namespace: genai-dev
│   └─ Helm release: genai-dev
└─ namespace: genai-prod
    └─ Helm release: genai-prod
```

Advantages:

- Lower cost.
- Easier initial setup.
- Shared observability infrastructure.

Disadvantages:

- Weaker failure isolation.
- More complicated resource and access controls.
- Development workloads could affect production.

Option B: Separate clusters or AWS accounts

```
Development AWS account
└─ genai-dev-eks

Production AWS account
└─ genai-prod-eks
```

Advantages:

- Stronger security and resource isolation.
- Easier production access control.
- Lower blast radius.

Disadvantages:

- More infrastructure and operational cost.
- Cross-account image and deployment access must be configured.

For an interview, a good response is:

```
"I may use namespace separation for an early-stage platform, but I prefer a separate production account and EKS cluster for a regulated or high-criticality GenAI service."
```

7. Simplified realistic Jenkinsfile

The following example uses:

- Pull-request testing.
- Main-branch Docker build and ECR push.
- Automatic development deployment.
- Post-deployment RAG evaluation.
- Optional manual production promotion.
- The same image tags in development and production.

```
pipeline {
    // This agent must contain:
    // Git, Python, Node.js, Docker, AWS CLI, kubectl and Helm.
    agent {
        label 'docker-aws-helm'
    }

    options {
        timestamps()

        // Prevent two deployments from changing the same Helm release
        // at the same time.
        disableConcurrentBuilds()

        // Retain only a reasonable amount of Jenkins build history.
        buildDiscarder(logRotator(numToKeepStr: '30'))

        timeout(time: 45, unit: 'MINUTES')
        skipDefaultCheckout(true)
    }

    parameters {
        booleanParam(
            name: 'PROMOTE_TO_PROD',
            defaultValue: false,
            description: 'After dev validation, allow this image to be
promoted to production'
        )
    }

    environment {
        AWS_REGION = 'us-east-1'

        // Replace these example values with your AWS resources.
        ECR_REGISTRY = '123456789012.dkr.ecr.us-east-1.amazonaws.com'
        BACKEND_REPO = 'rag-backend'
        FRONTEND_REPO = 'chat-frontend'

        EKS_CLUSTER_DEV = 'genai-dev-eks'
        EKS_CLUSTER_PROD = 'genai-prod-eks'

        HELM_CHART = 'deploy/helm/genai-app'
```

```

DEV_NAMESPACE = 'genai-dev'
PROD_NAMESPACE = 'genai-prod'

DEV_RELEASE = 'genai-dev'
PROD_RELEASE = 'genai-prod'

DEV_URL = 'https://dev.genai.example.com'
PROD_URL = 'https://genai.example.com'
}

stages {
  stage('Checkout') {
    steps {
      checkout scm
    }
  }

  stage('Initialize') {
    steps {
      script {
        // Example: 184-a91f72c315e4
        def shortSha = sh(
          script: 'git rev-parse --short=12 HEAD',
          returnStdout: true
        ).trim()

        env.IMAGE_TAG = "${env.BUILD_NUMBER}-${shortSha}"

        env.BACKEND_IMAGE =
          "${env.ECR_REGISTRY}/${env.BACKEND_REPO}:$
{env.IMAGE_TAG}"

        env.FRONTEND_IMAGE =
          "${env.ECR_REGISTRY}/${env.FRONTEND_REPO}:$
{env.IMAGE_TAG}"
      }

      echo "Backend image: ${env.BACKEND_IMAGE}"
      echo "Frontend image: ${env.FRONTEND_IMAGE}"
    }
  }

  stage('Lint and Unit Tests') {
    parallel {
      stage('Backend Tests') {
        steps {
          sh '''
            set -e

            cd backend

            python3 -m venv .venv
            . .venv/bin/activate

```

not

```
        pip install --upgrade pip
        pip install -r requirements-dev.txt

        ruff check .
        pytest -q \
            --junitxml=reports/pytest.xml
    ...
}

post {
    always {
        junit(
            testResults: 'backend/reports/pytest.xml',
            allowEmptyResults: true
        )
    }
}

stage('Frontend Tests') {
    steps {
        sh '''
            set -e

            cd frontend

            npm ci
            npm run lint

            # Define test:ci in package.json so tests do

            # remain in interactive/watch mode.
            npm run test:ci
            npm run build
        ...
    }
}

stage('Static RAG Golden Tests') {
    // Pull requests can run these tests too if they do not require
    // expensive external model calls.
    steps {
        sh '''
            set -e

            cd backend
            . .venv/bin/activate

            python scripts/run_golden_eval.py \
                --dataset ../evaluation/golden-dataset.jsonl \
                --mode mocked \
```

```

        ...        --min-retrieval-score 0.90
    }
}

stage('Build Docker Images') {
    // In a Multibranch Pipeline, PR branches stop before pushing.
    when {
        branch 'main'
    }

    steps {
        sh '''
            set -e

            docker build \
                --pull \
                --label "org.opencontainers.image.revision=${GIT_COMMIT}" \
                --label "org.opencontainers.image.version=${IMAGE_TAG}" \
                --tag "${BACKEND_IMAGE}" \
                backend

            docker build \
                --pull \
                --label "org.opencontainers.image.revision=${GIT_COMMIT}" \
                --label "org.opencontainers.image.version=${IMAGE_TAG}" \
                --tag "${FRONTEND_IMAGE}" \
                frontend

            ...
        '''
    }

stage('Push Images to ECR') {
    when {
        branch 'main'
    }

    // This demonstrates Jenkins credentials.
    // Prefer an IAM role on the Jenkins agent where possible.
    environment {
        AWS_ACCESS_KEY_ID =
            credentials('jenkins-aws-access-key-id')

        AWS_SECRET_ACCESS_KEY =
            credentials('jenkins-aws-secret-access-key')
    }

    steps {
        sh '''
            set -e

```

```

        aws ecr get-login-password \
            --region "${AWS_REGION}" |
        docker login \
            --username AWS \
            --password-stdin "${ECR_REGISTRY}"

        docker push "${BACKEND_IMAGE}"
        docker push "${FRONTEND_IMAGE}"

        docker logout "${ECR_REGISTRY}"
    ...
}
}

stage('Deploy to Dev') {
    when {
        branch 'main'
    }

    environment {
        AWS_ACCESS_KEY_ID =
            credentials('jenkins-aws-access-key-id')

        AWS_SECRET_ACCESS_KEY =
            credentials('jenkins-aws-secret-access-key')
    }

    steps {
        sh '''
            set -e

            export KUBECONFIG="${WORKSPACE}/.kubeconfig-dev"

            aws eks update-kubeconfig \
                --region "${AWS_REGION}" \
                --name "${EKS_CLUSTER_DEV}" \
                --kubeconfig "${KUBECONFIG}"

            # Fail before touching the cluster if the chart is
invalid.
            helm lint "${HELM_CHART}" \
                -f "${HELM_CHART}/values-dev.yaml"

            helm upgrade --install "${DEV_RELEASE}" "${HELM_CHART}"
\
                --namespace "${DEV_NAMESPACE}" \
                --create-namespace \
                -f "${HELM_CHART}/values-dev.yaml" \
                --set backend.image.repository="${ECR_REGISTRY}/${
{BACKEND_REPO}" \
                --set-string backend.image.tag="${IMAGE_TAG}" \
                --set frontend.image.repository="${ECR_REGISTRY}/${
{FRONTEND_REPO}" \

```

```

        --set-string frontend.image.tag="${IMAGE_TAG}" \
        --wait \
        --timeout 10m \
        --rollback-on-failure
    ...
}
}

stage('Dev Smoke Tests') {
    when {
        branch 'main'
    }

    steps {
        sh '''
            set -e

            # Retry because an Ingress/load balancer can require
            # additional time after the Kubernetes rollout
            success=0

            for attempt in $(seq 1 12); do
                if curl --fail --silent --show-error \
                    "${DEV_URL}/health"; then
                    success=1
                    break
                fi

                echo "Health check attempt ${attempt} failed"
                sleep 10
            done

            test "${success}" -eq 1

            # Basic end-to-end RAG smoke test.
            python backend/scripts/smoke_rag.py \
                --base-url "${DEV_URL}" \
                --question "What is the document retention policy?"
            ...
        '''
    }
}

stage('Dev RAG Quality Gate') {
    when {
        branch 'main'
    }

    steps {
        sh '''
            set -e

            cd backend
            . .venv/bin/activate

```

completes.

```

        # Tests the actual deployed application, including:
        # API, retriever, vector DB, prompts and LLM.
        python scripts/run_golden_eval.py \
            --base-url "${DEV_URL}" \
            --dataset ../evaluation/golden-dataset.jsonl \
            --mode live \
            --min-retrieval-score 0.90 \
            --min-answer-score 0.85 \
            --max-unsupported-rate 0.02
    ...
}
}

stage('Approve Production') {
    when {
        allOf {
            branch 'main'

            expression {
                return params.PROMOTE_TO_PROD
            }
        }
    }

    steps {
        input(
            message:
                "Promote image ${env.IMAGE_TAG} to production?",
            ok: 'Deploy to Production'
        )
    }
}

stage('Deploy to Production') {
    when {
        allOf {
            branch 'main'

            expression {
                return params.PROMOTE_TO_PROD
            }
        }
    }

    environment {
        // In a stronger setup, production would use a separate
        // Jenkins credential or assumed IAM role.
        AWS_ACCESS_KEY_ID =
            credentials('jenkins-prod-aws-access-key-id')

        AWS_SECRET_ACCESS_KEY =
            credentials('jenkins-prod-aws-secret-access-key')
    }
}

```



```

steps {
  sh '''
    set -e

    export KUBECONFIG="${WORKSPACE}/.kubeconfig-prod"

    aws eks update-kubeconfig \
      --region "${AWS_REGION}" \
      --name "${EKS_CLUSTER_PROD}" \
      --kubeconfig "${KUBECONFIG}"

    helm lint "${HELM_CHART}" \
      -f "${HELM_CHART}/values-prod.yaml"

    # Notice that IMAGE_TAG is unchanged.
    # Production receives the exact images tested in dev.
    helm upgrade --install "${PROD_RELEASE}" "$
{HELM_CHART}" \
      --namespace "${PROD_NAMESPACE}" \
      --create-namespace \
      -f "${HELM_CHART}/values-prod.yaml" \
      --set backend.image.repository="${ECR_REGISTRY}/$
{BACKEND_REPO}" \
      --set-string backend.image.tag="${IMAGE_TAG}" \
      --set frontend.image.repository="${ECR_REGISTRY}/$
{FRONTEND_REPO}" \
      --set-string frontend.image.tag="${IMAGE_TAG}" \
      --wait \
      --timeout 10m \
      --rollback-on-failure
    ...
  }
}

stage('Production Smoke Test') {
  when {
    allOf {
      branch 'main'

      expression {
        return params.PROMOTE_TO_PROD
      }
    }
  }

  steps {
    sh '''
      set -e

      curl --fail --silent --show-error \
        "${PROD_URL}/health"

      python backend/scripts/smoke_rag \

```

```

                                --base-url "${PROD_URL}" \
                                --question "What is the document retention policy?"
                                ...
                            }
                        }
                    }

    post {
        always {
            // Remove temporary cluster access files.
            sh '''
                rm -f "${WORKSPACE}/.kubeconfig-dev"
                rm -f "${WORKSPACE}/.kubeconfig-prod"
                ...
            '''
        }

        success {
            echo "Pipeline succeeded for image ${env.IMAGE_TAG}"
        }

        failure {
            echo 'Pipeline failed. Inspect the failed stage and Helm
history.'
        }
    }
}

```

Declarative Pipeline supports credentials binding, stage-specific environments and post conditions such as `always`, `success` and `failure`. Jenkins masks bound credentials in normal console output, but masking alone does not prevent malicious pipeline code from stealing them; trusted credentials must therefore never be exposed to untrusted jobs. ([Jenkins](#))

8. Secrets and credentials

There are three different secret categories.

Jenkins deployment credentials

These allow Jenkins to:

- Push to ECR.
- Describe an EKS cluster.
- Generate kubeconfig.
- Upgrade specific Helm releases.

Preferred order:

1. **IAM role attached to an EC2 Jenkins agent.**
2. **IAM role for a Jenkins agent running in EKS.**
3. **STS role assumption or workload identity.**
4. Static access keys in Jenkins Credentials as a fallback.

Jenkins documentation explicitly recommends AWS IAM roles over long-lived static AWS keys when Jenkins agents run in AWS. ([Jenkins](#))

Kubernetes deployment authorization

The Jenkins AWS identity should map to restricted Kubernetes permissions.

For example, allow Jenkins to operate only in:

```
genai-dev
genai-prod
```

Avoid giving the deployment identity:

```
cluster-admin
```

A production role might be restricted to:

- Reading Deployments, Pods and Services.
- Creating and updating namespaced application resources.
- Reading release status.
- Not accessing unrelated namespaces.

Runtime application secrets

Runtime secrets include:

```
OPENAI_API_KEY
ANTHROPIC_API_KEY
PINECONE_API_KEY
DATABASE_URL
REDIS_PASSWORD
JWT_SECRET
```

Do not pass these directly using:

```
helm upgrade --set openaiApiKey=...
```

That can expose values through:

- Jenkins process arguments.
- Build logs.
- Shell history.
- Helm release metadata.

A stronger design is:

```
AWS Secrets Manager
    ↓
External Secrets Operator or Secrets Store CSI driver
    ↓
Kubernetes Secret
```

↓
FastAPI pod environment or mounted file

The Helm values should generally contain the **secret reference**, not the secret value:

```
backend:
  existingSecret: rag-backend-runtime-secrets
```

9. Rollback strategies

Automatic rollback

Use Helm waiting and rollback behavior:

```
helm upgrade --install ... \
  --wait \
  --timeout 10m \
  --rollback-on-failure
```

This handles failures such as:

- Pods never becoming ready.
- Invalid configuration.
- Failed Kubernetes hooks.
- Readiness-probe failures.

Manual rollback

View release history:

```
helm history genai-prod -n genai-prod
```

Rollback to a specific revision:

```
helm rollback genai-prod 12 \
  -n genai-prod \
  --wait
```

When the revision is omitted or set to zero, Helm rolls back to the previous release. Helm records incremental release revisions for installs, upgrades and rollbacks. (helm.sh)

Roll back using an image tag

Because every image has an immutable tag, you can also redeploy a known-good image:

```
helm upgrade genai-prod deploy/helm/genai-app \
  -n genai-prod \
  -f deploy/helm/genai-app/values-prod.yaml \
  --set-string backend.image.tag=178-65bc820a91fe \
  --set-string frontend.image.tag=178-65bc820a91fe
```

This is useful when:

- The chart itself is correct.
- Only the application image is faulty.
- You want an explicit GitOps-style image rollback.

Database rollback caution

Application rollback is easy only when database and index changes are backward-compatible.

Use patterns such as:

```
Expand schema
Deploy compatible application
Migrate data
Remove old schema later
```

Avoid deploying an irreversible database migration and assuming that a Helm rollback will automatically repair the database.

The same caution applies to:

- Vector-index schema changes.
- Embedding-model changes.
- Metadata format changes.
- Prompt-template contracts.
- Conversation-state schemas.

10. Promotion strategy

A strong promotion workflow is:

```
Pull request
  ↓
Lint + unit tests + mocked RAG evaluation
  ↓
Merge to main
  ↓
Build image once
  ↓
Push immutable image to ECR
  ↓
Deploy to development
  ↓
Live golden tests and smoke tests
  ↓
Manual or policy-based approval
  ↓
Deploy same image digest to production
```

Do not use:

Build dev image → test → rebuild prod image

Even from the same commit, dependency downloads, base-image changes or build timestamps could produce a different artifact.

For stricter production controls, save promotion metadata:

Git SHA
Jenkins build number
Backend image digest
Frontend image digest
Helm chart version
Golden-test result
Approver
Deployment timestamp

11. GenAI-specific CI/CD checks

Retrieval regression

Validate that important questions still retrieve the correct documents after:

- Changing embedding models.
- Modifying chunk size.
- Changing overlap.
- Adding a reranker.
- Updating metadata filters.
- Re-indexing documents.

Prompt regression

Check whether prompt changes cause:

- Missing citations.
- Longer answers.
- Unsupported claims.
- Incorrect tool calls.
- Failure to refuse unanswerable questions.

Safety regression

Test prompts involving:

- Prompt injection.
- Requests to reveal system instructions.
- Requests for unauthorized documents.
- PII extraction.
- Cross-tenant data access.
- Malicious content embedded in retrieved documents.

Performance regression

Capture:

```
Retrieval latency
Time to first token
End-to-end latency
Input and output tokens
LLM cost per request
Vector DB query time
Error rate
```

A release may be semantically correct but still unsuitable for production if latency or cost increases significantly.

12. Best practices

1. Use ephemeral or dedicated agents

Do not build Docker images on the Jenkins controller. Use dedicated EC2 agents or dynamically created Kubernetes agents.

2. Pin tool versions

Pin versions of:

```
Python
Node.js
Docker build tooling
AWS CLI
kubectl
Helm
```

This reduces “works on one agent but fails on another” problems.

3. Use immutable image tags

Prefer:

```
build-number + commit-SHA
```

Also capture the final ECR digest.

4. Build once and promote

Development and production should deploy the same image digest.

5. Separate application and deployment permissions

The Jenkins deployment role does not need broad infrastructure-administration permissions.

For example, it should not generally need permission to:

- Delete the EKS cluster.
- Modify the VPC.

- Change IAM roles.
- Delete ECR repositories.

6. Validate Helm before deployment

Run:

```
helm lint
helm template
```

Optionally validate the rendered resources using policy or schema tools.

7. Use readiness probes

Helm's `--wait` is only useful when Kubernetes resources have meaningful readiness conditions.

For the RAG backend:

```
/health → process is alive
/ready → required dependencies are reachable
```

Do not make a liveness check depend on every external LLM provider, because a temporary provider outage could restart otherwise healthy pods continuously.

8. Add post-deployment verification

A successful `helm upgrade` does not prove that the application works from the user's perspective.

Run:

- Public health endpoint check.
- Authentication check.
- Simple chat query.
- Retrieval and citation check.
- Frontend availability check.

9. Protect production approval

Restrict who may approve production promotion. For higher-risk releases, require separation between the person who commits and the person who approves.

10. Move repeated logic to a Shared Library

As multiple applications adopt the same pipeline, move common functions such as ECR login, Helm deployment and smoke testing into a Jenkins Shared Library. Jenkins supports centrally managed Shared Libraries to reduce duplicated pipeline logic. ([Jenkins](#))

13. Common pitfalls

Pitfall	Why it is dangerous	Better approach
Deploying latest	Cannot identify or reproduce deployments	Commit-based tag and digest
Static AWS keys everywhere	Long-lived credential exposure	IAM roles and short-lived credentials

Building on controller	Security and scalability risk	Dedicated agents
Passing secrets with Helm <code>--set</code>	Secrets may appear in metadata or logs	Secrets Manager plus secret references
Rebuilding for production	Production artifact differs from tested artifact	Promote same image
No deployment timeout	Pipeline may wait indefinitely	<code>--wait</code> and <code>--timeout</code>
No readiness probe	Helm cannot verify application readiness	Proper readiness endpoint
Unit tests only	RAG quality regressions remain undetected	Golden and live evaluation
Same production and development role	Excessive blast radius	Separate IAM roles
No smoke test	Kubernetes success may hide application failure	API and RAG smoke tests
Automatic production on every merge	Risky for high-impact services	Approval or policy gate
Golden tests calling live LLMs on every PR	Expensive and flaky	Mocked PR tests; live pre-production gate
Environment-specific frontend rebuilds	Breaks build-once promotion	Runtime configuration where possible
Non-compatible database migration	Helm rollback cannot restore data	Expand-and-contract migrations

14. Interview Q&A

1. What is the difference between a Jenkins controller and agent?

The controller schedules pipelines, manages configuration, credentials and build history. Agents execute the actual commands such as tests, Docker builds and Helm deployments.

2. Why would you choose Declarative Pipeline?

Declarative Pipeline provides a structured and readable syntax with built-in support for stages, agents, conditions, credentials, parameters and post-build handling. Scripted Pipeline is more flexible but is usually harder to maintain.

3. How would you tag Docker images?

I would use an immutable tag containing the Jenkins build number and Git commit SHA, such as:

```
184-a91f72c315e4
```

I would also capture the ECR image digest for deployment traceability.

4. How does Jenkins authenticate with ECR?

The Jenkins agent uses an IAM role or AWS credentials. It runs:

```
aws ecr get-login-password |  
docker login --username AWS --password-stdin <registry>
```

The IAM principal must have least-privilege ECR push permissions.

5. How does Jenkins deploy to EKS?

Jenkins obtains cluster access using:

```
aws eks update-kubeconfig
```

It then runs `helm upgrade --install` using environment-specific values files and immutable image tags.

6. How do you promote a release from development to production?

I build the images once, push them to ECR, deploy them to development, run smoke and golden tests, obtain production approval and deploy the exact same image tags or digests to production.

7. How do you handle secrets?

Jenkins deployment credentials are stored in Jenkins Credentials or provided through IAM roles. Application runtime secrets remain in a dedicated secrets system such as AWS Secrets Manager and are injected into Kubernetes at runtime. I do not store secrets in Git, Docker images or normal Helm values.

8. How would you roll back a failed Helm release?

I would first use automatic rollback behavior during deployment. For a manual rollback:

```
helm history <release> -n <namespace>  
helm rollback <release> <revision> -n <namespace> --wait
```

Because images have immutable tags, I can also explicitly redeploy a previous known-good image.

9. What additional CI checks are needed for a RAG application?

In addition to unit tests, I would run retrieval tests, golden-question evaluations, citation checks, groundedness checks, prompt-injection tests, latency checks and post-deployment end-to-end smoke tests.

10. Why might a Helm deployment succeed while the application is still broken?

Kubernetes might successfully create the resources even though the application has incorrect configuration, inaccessible dependencies, broken authentication or poor RAG responses. Proper readiness probes and post-deployment API and golden tests are therefore required.

15. Interview-ready summary

“I would store a Declarative Jenkinsfile with the application source and run builds on dedicated agents. Pull requests execute backend and frontend linting, unit tests and mocked RAG golden tests. After merge, Jenkins builds immutable backend and frontend images, tags them with the build number and commit SHA, and pushes them to ECR using a least-privilege IAM identity. Jenkins then generates temporary EKS kubeconfig access and deploys to development using `helm upgrade --install` with environment-specific values. After readiness, smoke and live RAG-quality gates pass, an approved pipeline promotes the exact same image digests to production. Helm waiting, rollback behavior, previous image tags and release history support recovery, while runtime LLM and database secrets remain outside Jenkins and Helm values.”